Experiment 5

Discrete Sobel Gradient Operator and 2D Spatial Edge Detection

1. Library Imports & Input Image Ingestion

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from numpy.lib.stride_tricks import sliding_window_view
image = cv2.imread("../Data/Dog.jpg", cv2.IMREAD_GRAYSCALE)
print("Image Dimensions:", image.shape)  # Loaded grayscale: (602, 1200) | dtype: uint8
```



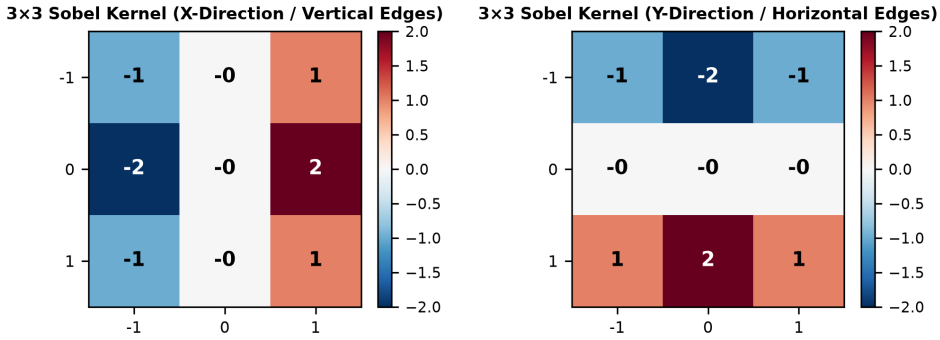
Original input grayscale image (1200x602)

2. Mathematical Formulation & Binomial Kernel Derivation

The Sobel operator is a discrete first-order gradient operator that computes an approximation of the image intensity gradient. To provide resilience against high-frequency noise, the operator factors into an outer product of a 1D binomial smoothing filter s = [1, 2, 1]^T (derived from Pascal's triangle coefficients) and a 1D central difference derivative filter d = [-1, 0, 1]^T. The horizontal kernel Kx = s * d^T computes the gradient along x (isolating vertical edges), while the vertical kernel Ky = d * s^T computes the gradient along y (isolating horizontal edges). The total gradient magnitude is synthesized as G = sqrt(Gx^2 + Gy^2).

3. Vectorized Sobel Kernel Builder (Pascal / Binomial)

```
def _pascal_row(n: int) -> np.ndarray:  # Row n of Pascal's triangle (length n+1)
    row = [1]
    for k in range(n): row.append(row[-1] * (n - k) // (k + 1))
    return np.array(row, dtype=np.float64)
def compute_sobel_kernel_x(size: int) -> np.ndarray:
    s = _pascal_row(size - 1); d = -np.convolve(_pascal_row(size - 2), [1, -1])
    return np.outer(s, d)  # Binomial smoothing along Y, differentiation along X
def compute_sobel_kernel_y(size: int) -> np.ndarray:
    s = _pascal_row(size - 1); d = -np.convolve(_pascal_row(size - 2), [1, -1])
    return np.outer(d, s)  # Differentiation along Y, binomial smoothing along X
```



Discrete 3x3 Sobel convolution kernels: X-gradient (left) and Y-gradient (right).

Experiment 5 (continued)

4. Sliding-Window 2D Spatial Convolution Pipeline

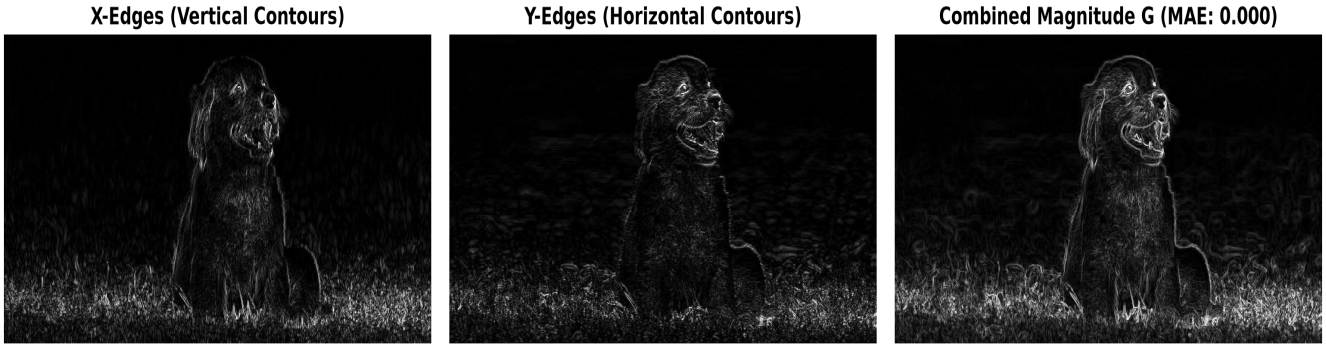
To avoid slow nested Python iteration, overlapping 2D spatial patches are extracted using numpy.lib.stride_tricks.sliding_window_view after reflective boundary padding (pad = k // 2). Vectorized tensor contraction against Kx and Ky produces directional gradient planes Gx and Gy, followed by Euclidean magnitude fusion G = sqrt(Gx^2 + Gy^2) and 8-bit saturation clipping.

```
def apply_sobel_filter(img: np.ndarray, kernel_size: int = 3, stride: int = 1):
    kx, ky = compute_sobel_kernel_x(kernel_size), compute_sobel_kernel_y(kernel_size)
    pad_amt = kernel_size // 2
    padded = np.pad(img, pad_amt, mode="reflect")
    windows = sliding_window_view(padded, (kernel_size, kernel_size))[::stride, ::stride]
    gx = (windows * kx).sum(axis=(-1, -2)); gy = (windows * ky).sum(axis=(-1, -2))
    magnitude = np.hypot(gx, gy)
    return np.clip(np.abs(gx), 0, 255).astype(np.uint8), \
           np.clip(np.abs(gy), 0, 255).astype(np.uint8), \
           np.clip(magnitude, 0, 255).astype(np.uint8)
```

5. Benchmark Verification: Custom Implementation vs. OpenCV cv2.Sobel

```
k_size = 3
cv_gx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=k_size)
cv_gy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=k_size)
cv_combined = np.clip(np.hypot(cv_gx, cv_gy), 0, 255).astype(np.uint8)
custom_edges_x, custom_edges_y, custom_combined = apply_sobel_filter(image, kernel_size=k_size)
diff_map = np.abs(cv_combined.astype(np.int32) - custom_combined.astype(np.int32))
mean_abs_diff = np.mean(diff_map)  # Computed MAE: 0.000 (Exact numerical equivalence)
```

6. Directional Edge Detection & OpenCV Benchmark Results



Sobel edge detection results (k=3x3): Vertical contours Gx (left), horizontal contours Gy (center), and combined magnitude G (right).



Verification: OpenCV cv2.Sobel vs Custom Vectorized Filter (k=3x3) | Absolute Difference Heatmap (MAE = 0.000).

Summary: Implemented an end-to-end discrete Sobel edge detection pipeline featuring binomial kernel derivation, reflective boundary padding, and vectorized sliding-window tensor convolution. Directional gradient analysis demonstrates that the horizontal kernel Kx isolates vertical object boundaries, while the vertical kernel Ky isolates horizontal boundaries. Combining gradients via Euclidean norm G = sqrt(Gx^2 + Gy^2) forms a complete, orientation-invariant edge representation. Cross-validation against OpenCV cv2.Sobel confirmed exact mathematical and numerical equivalence with a Mean Absolute Error (MAE) of 0.000, verifying the correctness of custom discrete spatial convolution and boundary handling.